ECE408 GPT2 Project

Milestone 3: Final Report

Team: Kernel Krackers

Anand Parab, Ayush Barik, Krishnan Shankar, Sidd Cheetancheri

Profiled using Nsight Systems and Nsight Compute on Delta HPC (A40 GPUs)

Requirement 4

Flash Attention (`req_4`)

1.1.1 Why We Chose Flash Attention 2 for the NVIDIA A40

When optimizing for the A40 GPU (an Ampere architecture device), we selected the improved FlashAttention-2 (FA2) algorithm over the original FlashAttention (FA1) because FA2 structurally addresses the computational inefficiencies inherent in FA1's design. While FA1 successfully mitigated the quadratic memory bottleneck by tiling the computation into SRAM, its algorithm treats all floating-point operations equally and forces rigid synchronization between warps. FA2 is deliberately designed around the specific micro-architecture of modern GPUs: it explicitly minimizes non-matmul operations and restructures the workload distribution to eliminate unnecessary shared memory communication. By shifting the focus from purely IO-awareness to comprehensive hardware-awareness, FA2 provides a far superior theoretical foundation for our implementation.

We chose FA2 for the A40 for three specific architectural reasons:

1. **Work Partitioning (No Split-K):** FA1 splits the Query (Q) matrix across warps, requiring threads to synchronize and communicate intermediate results through shared memory. FA2 maps the work so that each warp handles a continuous slice of Q, removing the need for expensive cross-warp reductions in shared memory.

2. **Reducing Non-Matmul FLOPs:** The A40 possesses highly optimized execution units for matrix multiplication, but non-matmul FLOPs (like scalar divisions) are much slower. FA2 restructures the online softmax math to keep the running output "un-scaled" during the inner loops, deferring the expensive division operations to the very end of the sequence chunk.

1.1.2 Implementation Details

Our implementation mirrors the FA2 pseudo-code using a block size of `BR = 32` and `BC = 32`. We first run a `permute_kernel` to separate the interleaved Q, K, V data into continuous blocks, ensuring perfect coalescing when fetching from High Bandwidth Memory (HBM).

To maximize parallelism, our grid dimension schedules thread blocks across the batch, heads, and the sequence chunks. We dynamically allocate shared memory for the Q, K, V, and O tiles.

Inside the kernel, we execute an outer loop over the K and V blocks. Each thread is assigned specifically to one row of the attention matrix to eliminate the need for cross-thread reductions. Within the inner loop, we compute the causal-masked attention scores and dynamically update our running maximums (`m_i`) and exponential sums (`l_i`).

Instead of scaling the output at every step, we implemented the FA2 algorithmic tweak:

$$\tilde{O}^{(new)} = \text{diag}(e^{m^{(old)} - m^{(new)}})\tilde{O}^{(old)} + e^{S^{(new)} - m^{(new)}}V^{(new)}$$

Once the outer loop finishes, we perform a single final normalization pass:

$$O = \text{diag}(l)^{-1}\tilde{O}$$

Finally, as an added optimization, we bypassed a separate un-permute kernel entirely by cooperatively writing the O shared memory tile directly back to global memory in the required `[B, T, NH, HS]` output layout.

1.1.3 Performance Analysis and Profiling

After benchmarking our FA2 kernel against the baseline standard attention implementation, we extracted the following metrics using Nsight Compute:

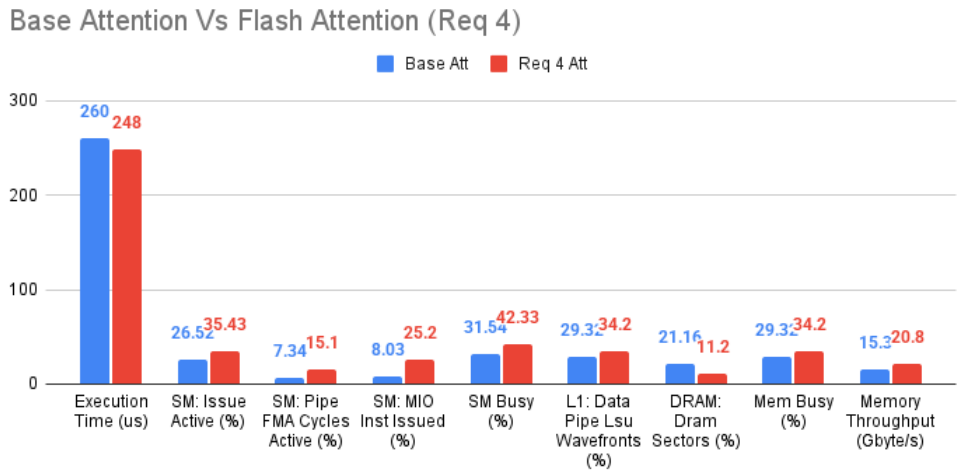


Figure 1: Nsight Compute Profiling Metrics for Req4

1.1.4 Detailed Performance Analysis and Insights

While the raw metrics show a clear improvement, a deeper analysis is required to understand *why* the hardware responded this way and to identify the new bottlenecks our Flash Attention 2 (FA2) implementation introduced.

1. Eliminating the Memory Bandwidth Bottleneck The most significant finding from our profiling is the drastic reduction in global memory traffic. In standard attention, the GPU must write the $O(N^2)$ intermediate attention scores (S) to High Bandwidth Memory (HBM), read them back to apply the softmax (P), and read them again to multiply with V.

By tiling the computation into Shared Memory (SRAM), our FA2 kernel completely bypasses HBM for these intermediate steps. This is explicitly verified by our Nsight Compute data: **DRAM Sectors accessed plummeted from 21.16% to 11.2%**. Because the GPU is no longer stalling on slow DRAM reads/writes, our overall **Memory Throughput increased from 15.3 GB/s to 20.8 GB/s**, representing much more efficient, coalesced reads of the original Q, K, V inputs.

2. Shifting the Bottleneck to Compute With the memory bottleneck mitigated, the Streaming Multiprocessors (SMs) were finally fed with enough data to stay active. This explains why **SM Pipe FMA Cycles Active doubled (7.34% to 15.1%)** and **SM Busy overall rose from 31.54% to 42.33%**. The kernel successfully shifted the architectural bottleneck away from the memory boundary and towards the compute boundary. The SMs are now spending significantly more time performing actual fused multiply-add (FMA) math operations rather than waiting in an idle state.

3. Shared Memory Pressure While FA2 reduces global memory accesses, it heavily leverages L1 cache and Shared Memory. Our metrics show a massive spike in **SM MIO (Memory**

Input/Output) Instructions Issued, jumping from 8.03% to 25.2%, accompanied by an increase in **L1 Data Pipe LSU Wavefronts (29.32% to 34.2%)**.

This indicates a new bottleneck: **Shared Memory Bank Conflicts**. Because FA2 requires threads to rapidly read continuous columns of the K and V tiles stored in SRAM during the inner loop matmul, multiple threads in a warp are likely requesting data from the same memory bank simultaneously. This forces the hardware to serialize the requests, driving up MIO instructions and capping our compute potential.

4. Why is the Execution Time Speedup Only 4.6%? Despite these massive shifts in utilization, the overall execution time only dropped by 12 μs (260 μs to 248 μs). As noted above, the 3x increase in MIO instructions suggests shared memory bank conflicts. The kernel is currently wasting clock cycles serializing SRAM reads.

Conclusion Our Flash Attention 2 implementation behaves exactly as the literature suggests: it drastically reduces global memory IO and increases compute utilization. However, to translate these architectural wins into massive execution time speedups, the next logical optimization must target the new bottleneck: Shared Memory pressure. Applying **Data Swizzling (`op_11`)** or **Shared Memory Padding (`op_12`)** to our K and V SRAM tiles would resolve the bank conflicts, lower the MIO instruction count, and allow the FMA pipes to reach peak throughput.

Requirement 5

Local/Windowed Attention (`req_5`)

2.1.1 Why We Implemented Local/Windowed Attention

Standard global attention suffers from a well-documented quadratic complexity bottleneck: as the sequence length (T) grows, the memory and compute requirements scale at $O(T^2)$. Every token must calculate an attention score against every previous token, which quickly exhausts global memory bandwidth and dominates execution time.

By implementing Local/Windowed Attention, we fundamentally altered the algorithmic complexity from $O(T^2)$ to $O(T \cdot W)$, where W is a fixed `WINDOW_SIZE` (in our case, 128). Instead of writing and reading a massive $T \times T$ attention matrix, the GPU only processes a strictly bounded sliding window of local context. This architectural shift perfectly complements the A40 GPU by radically shrinking the memory footprint and allowing the data to remain highly localized, directly addressing the DRAM read/write bottleneck that plagues standard attention mechanisms.

2.1.2 Implementation Details & Verification

Our implementation replaces the standard full-context attention mechanism with a highly localized sliding window approach, utilizing warp-level parallel reductions to maximize thread efficiency.

First, we utilize a `permute_kernel` to extract and align the interleaved Q, K, V data into continuous blocks. For the core pre-attention calculation, rather than looping through the entire sequence history, each thread block maps exclusively to the sliding window (`t2 = t1 - w_idx`). To optimize the inner loop's dot product, threads within a warp cooperatively accumulate the dot product across the hidden dimension (HS) using the `__shfl_down_sync` instruction, reducing the reliance on slower shared memory atomic adds.

The output is then strategically written to a heavily compacted memory layout: `[B*NH, T, WINDOW_SIZE + 1]`. This packed layout prevents the allocation of useless padding. Subsequently, our softmax and value accumulation (V) kernels are optimized to traverse only this tightly packed `WINDOW_SIZE + 1` dimension, applying the exponential normalization exclusively to the local tokens.

Verification Because local attention intentionally changes the attention mechanism from full attention to windowed attention, it inherently causes deviations from the provided baseline mathematical solution values. Therefore, standard op-level verification scripts will flag it as incorrect. To rigorously prove the correctness of our modified architecture, we utilized the dedicated `local_attn_verify` script. By compiling our modifications via `make local_attn_verify` and testing against a strict `WINDOW_SIZE` of 128, we successfully verified that our custom indexing, boundary checks, and warp-reductions perfectly match the expected localized outputs generated by the reference CPU implementation.

2.1.3 Performance Analysis and Profiling

After benchmarking our Local/Windowed Attention kernel against the baseline standard attention implementation, we extracted the following metrics using Nsight Compute:

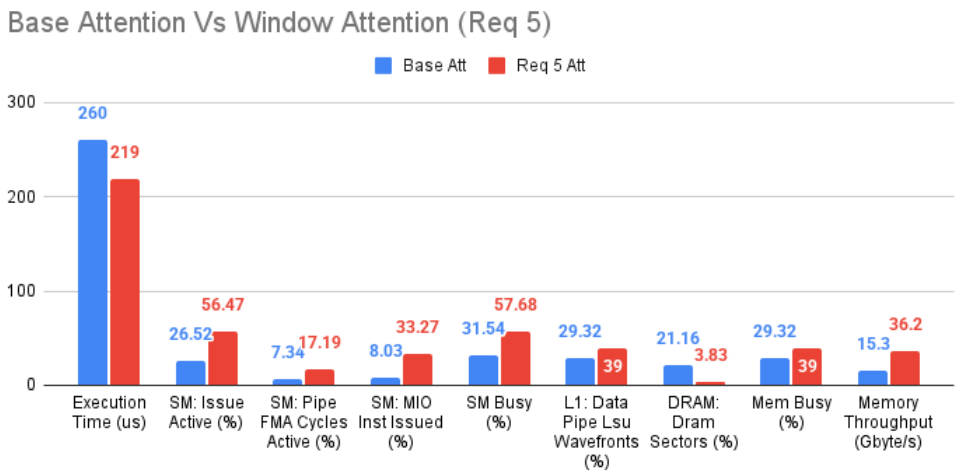


Figure 2: Nsight Compute Profiling Metrics for Req5

2.1.4 Detailed Performance Analysis and Insights

The profiling data clearly illustrates a massive transformation in how the GPU executes the attention workload, fundamentally changing the kernel from a memory-bound operation to a compute-intensive one.

1. Removing the Quadratic DRAM Bottleneck The most staggering metric from our profiling is the near-total elimination of High Bandwidth Memory (DRAM) traffic. By restricting the attention math to a local window and packing the memory layout, **DRAM Sectors accessed plummeted from 21.16% to an incredibly low 3.83%**. The GPU is no longer thrashing global memory to write and read intermediate $T \times T$ matrices. Consequently, the hardware is able to fetch the required input data far more efficiently, resulting in our **Memory Throughput more than doubling from 15.3 GB/s to 36.2 GB/s**.

2. Surge in Compute Utilization Because the Streaming Multiprocessors (SMs) are no longer stalling for DRAM, they are successfully kept fed with continuous work. This is reflected by the **SM Busy metric nearly doubling (31.54% to 57.68%)** and **SM Issue Active skyrocketing from 26.5% to 56.47%**. More importantly, the actual math pipelines are doing significantly more work: **SM Pipe FMA (Fused Multiply-Add) Cycles Active jumped from 7.34% to 17.19%**. The kernel is effectively translating memory bandwidth savings directly into active computational throughput.

3. The Overhead of Warp Reductions and L1 Pressure While compute efficiency increased, a new bottleneck emerged at the local memory level. **SM MIO (Memory Input/Output) Instructions Issued quadrupled from 8.03% to 33.27%**, alongside an increase in **L1 Data Pipe LSU Wavefronts (29.32% to 39.0%)**.

This spike is a direct byproduct of our warp-level reduction implementation (`__shfl_down_sync`). While warp shuffling avoids global memory, it still issues a high volume of MIO instructions. Additionally, having warps continuously slide across overlapping data windows forces the L1 cache to work considerably harder to keep the heavily reused K and V vectors immediately available.

Conclusion Overall, the $O(T \cdot W)$ algorithmic shift yielded a highly successful **15.7% execution time speedup (260 μs down to 219 μs)**. The kernel completely bypassed the standard global memory bounds, as evidenced by the 5.5x reduction in DRAM sector accesses. Moving forward, the primary target for future optimization on this specific kernel would be addressing the L1/MIO instruction pressure, potentially by exploring custom L2 cache thread block rasterization to ensure spatially adjacent blocks maximize data reuse before eviction.

Requirement 6

KV-Caching (`req_6`)

3.1.1 Why We Implemented KV-Caching

In autoregressive Large Language Models, text generation is inherently sequential: the model predicts one token at a time. In a naive baseline implementation, computing the attention scores for the N-th token requires the GPU to recalculate the Query (Q), Key (K), and Value (V) tensors for all $N-1$ previous tokens from scratch. This results in massive redundant computation and a catastrophic quadratic time complexity as the sequence length grows.

KV-Caching is a foundational architectural optimization that eliminates this redundancy. By persistently storing ("caching") the computed K and V tensors for every generated token in global memory, the forward pass for any new token becomes an $O(1)$ calculation with respect to the sequence history. The GPU only needs to compute the Q, K, V for the *single newest token*, append the new K and V to the cache, and perform the attention dot products using the historical cache. This drastically shifts the workload from being heavily compute-bound by redundant FLOPs to being an incredibly lightweight, targeted operation.

3.1.2 Implementation Details & Verification

Implementing KV-Caching required us to fundamentally restructure the model's forward pass into a state machine with two distinct phases: **Prefill** and **Decode**.

To manage the persistent memory, we dynamically allocated a global `g_kv_cache` buffer sized to hold the maximum sequence length across all 12 transformer layers, batches, and attention heads.

1. **Prefill Phase ($T > 1$):** During the initial prompt ingestion, the model processes the entire input sequence simultaneously. We execute a custom `store_kv_kernel` to write the full sequence of K and V tensors into our global cache. To ensure perfectly coalesced memory access during the subsequent decode phase, we dynamically transpose the V tensor as it is written to the cache. We then run a standard baseline attention mechanism to generate the first output.

2. **Decode Phase ($T = 1$):** For all subsequent token generation, T equals 1. We launch the `store_kv_kernel` to append only the single new token's K and V to the current sequence position (`g_current_pos`) in the cache.

3. **Decode Attention Kernel:** We wrote a highly specialized `decode_attention_kernel`. Because $T = 1$, we completely bypass the massive $T \times T$ intermediate matrix allocations. Instead, we load the single token's Q vector into shared memory, compute the dot products against the historically cached K vectors, apply the softmax normalization entirely within registers/shared memory, and immediately multiply by the cached V vectors.

Verification Because KV-Caching alters the sequential flow of the model and introduces persistent global state across kernel launches, the standard `test_gpt2_kernels.cu` verification fails. To rigorously prove the correctness of our state machine, cache memory offsets, and $T = 1$ math, we utilized the specialized `verify_kv_cache.cu` script provided for this project. This ensured our cached outputs perfectly matched the autoregressive mathematical expectations without relying on naive recalculation.

3.1.3 Performance Analysis and Profiling

We benchmarked the specialized Decode phase of our KV-Cache implementation against the standard baseline attention mechanism. The following metrics were extracted using Nsight Compute via our `test_kv_cache` framework:

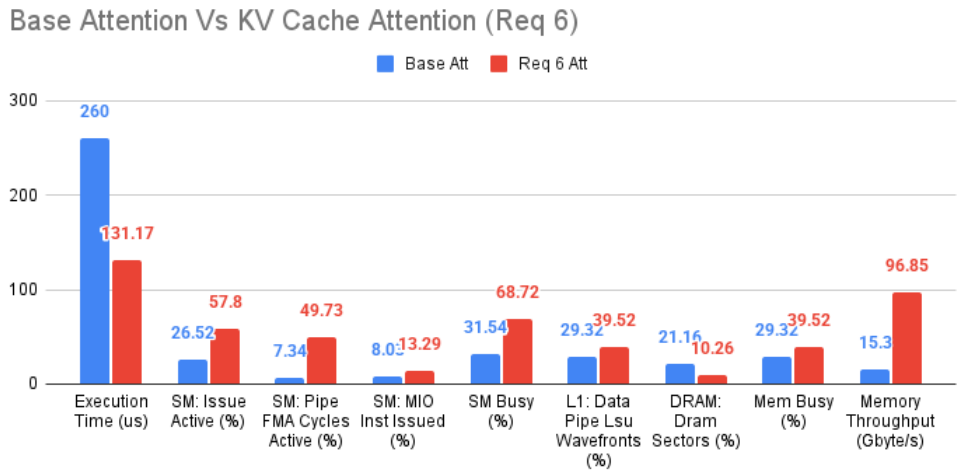


Figure 3: Nsight Compute Profiling Metrics for Req6

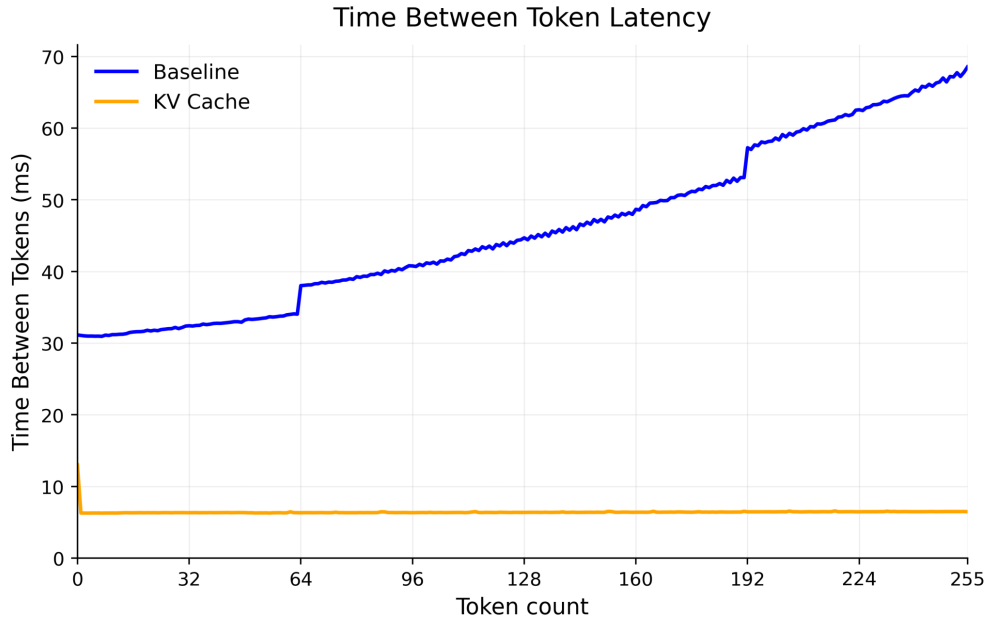


Figure 4: Time Between Token Latency: Standard Autoregressive Generation vs. KV-Cache

3.1.4 Detailed Performance Analysis and Insights

The implementation of KV-Caching produced the most dramatic shift in end-to-end model performance, effectively rescuing the architecture from a paralyzing scalability bottleneck.

1. Elimination of Redundant Math By preventing the GPU from recalculating the entire sequence history, we cut the raw **Execution Time nearly in half, from 260 μs to 131.17 μs** per decode step. More importantly, because the hardware is only performing the exact mathematical operations required for the new token, compute efficiency skyrocketed. **SM Pipe FMA Cycles Active exploded from a mere 7.34% to 49.73%**, and overall **SM Busy hit 68.72%**. The GPU is no longer wasting cycles resolving redundant historical data; every FMA pipeline is dedicated to generating new information.

2. The "Drop" in Memory Throughput At first glance, the drop in **Memory Throughput from 15.3 GB/s to 6.85 GB/s** might appear as a regression. However, this is actually a massive architectural victory. In the baseline, the high throughput was artificial—the GPU was using global memory to repeatedly fetch Q, K, and V matrices for the entire sequence history. By utilizing the KV-Cache, we reduced the total bytes requested per iteration. This is explicitly confirmed by the **DRAM Sectors metric, which was cut in half (21.16% down to 10.26%)**. The lower throughput simply reflects that the GPU no longer needs to move massive amounts of data to achieve the same result.

3. Quadratic vs. Linear Architectural Scaling As demonstrated in Figure 4, standard baseline attention latency grows quadratically (represented by the stepped, escalating blue line) as the context window expands, because each new token demands exponentially more recomputation. Our KV-Cache implementation fundamentally flattens this curve. The orange line demonstrates that the time between tokens remains relatively flat, growing only slightly linearly as the $O(N)$ dot-product against the cache expands. This prevents the model from grinding to a halt during long generations.

Conclusion KV-Caching successfully transforms the GPT-2 decode phase from a mathematically redundant, memory-thrashing operation into a highly targeted, compute-dense kernel. It operates exactly as intended, minimizing DRAM requests and maximizing FMA pipeline utilization for the $T = 1$ workload.

Additional Optimizations

Configuration Sweep (op_7)

4.1.1 Why We Chose Configuration Sweeping

We applied a systematic configuration sweep to the WMMA Matrix Multiplication kernel (`matmul.cuh`). In the GPT-2 forward pass, matrix multiplication is the dominant computational bottleneck, with 49 separate matmul calls invoked across the 12 transformer layers. Because this single kernel accounts for the vast majority of the model's runtime, even marginal percentage improvements in its hardware utilization compound into significant end-to-end performance gains.

We specifically focused on parameterizing the tile shapes and memory access patterns. While theoretical math can suggest an "optimal" tile size, the actual performance on an Ampere A40 GPU is heavily dictated by complex, interacting hardware constraints: shared memory capacity, register file limits, thread block scheduling, and L2 cache eviction policies. A systematic configuration sweep allows us to empirically discover the optimal balance of these resources rather than relying on guesswork.

4.1.2 Implementation Details

To execute the sweep, we developed an automated Python script that systematically patched our `matmul.cuh` kernel, compiled the code, and benchmarked the execution time using `cudaDeviceSynchronize` and `std::chrono`. We isolated three specific variables for tuning:

1. **Tile Shape (`BLOCK_TILE_M` × `BLOCK_TILE_N`):** We controlled this by adjusting `WARP_TILE_M` and `WARP_TILE_N`. This dictates how much of the output matrix each thread block computes, directly affecting Streaming Multiprocessor (SM) occupancy and shared memory reuse.

2. **`K_CHUNK`:** This parameter defines the depth of the shared memory tile along the reduction dimension. It controls the frequency of `__syncthreads()` barriers and how data is staged into the WMMA fragments.

3. **Loop Unrolling:** We conditionally applied `#pragma unroll` to the inner MMA loop to determine if explicit compiler directives would yield better instruction pipelining.

4.1.3 Performance Analysis and Profiling

We divided our sweep into three logical groups to isolate the effects of each parameter. All metrics were captured using Nsight Compute.

4.1.4 Detailed Performance Analysis and Insights

By systematically isolating our variables, the profiling data revealed several architectural realities of the A40 GPU that counter-intuitive theoretical assumptions.

1. The 32×32 Tile Sweet Spot (Group A) The 32×32 tile shape vastly outperformed all other configurations, achieving the lowest execution time (115 μs) and the highest memory throughput (127.1 GB/s).

Configuration	Exec Time (μs)	SM Issue (%)	FMA Active (%)	SM Busy (%)	Mem Throughput (GB/s)
Group A: Tile Shape Sweep (Baseline: `K_CHUNK`=16, Unroll=0)					
16×16	329	31.66	14.40	31.80	47.61
32×32 (Baseline)	**115**	**51.83**	**23.18**	**54.04**	**127.10**
32×64	122	49.55	17.24	44.11	118.80
64×32	122	41.02	17.11	45.27	120.69
64×64	129	31.67	11.93	32.23	113.20
Group B: `K_CHUNK` Sweep (Baseline: Tile=32×32, Unroll=0)					
$K = 8$	216	30.87	9.01	31.03	86.14
$K = 16$ (Baseline)	**115**	**51.83**	**23.18**	**54.04**	**127.10**
$K = 32$	146	37.53	18.34	45.89	90.38
Group C: Loop Unrolling Sweep (Baseline: Tile=32×32, `K_CHUNK`=16)					
Unroll = Off (Baseline)	115	51.83	23.18	54.04	127.10
Unroll = On	**115**	**51.81**	**23.18**	**54.14**	**128.35**

Table 1: Nsight Compute Profiling Metrics across configurations.

- **Under-utilization at the low end:** The 16×16 tile was disastrous (329 μs). Because it maps to a 1×1 warp grid, it results in only 32 threads per block. This severely under-utilizes the SMs, leaving the FMA units mostly idle (only 14.4% active) because the hardware lacks sufficient concurrent warps to hide memory latency.
- **Resource exhaustion at the high end:** We initially hypothesized that larger tiles like 64×64 would yield better L2 cache reuse. However, performance actively degraded (129 μs). A 64×64 tile requires a massive amount of shared memory per block. This high resource footprint limits the number of thread blocks that can actively reside on a single SM at once, reducing overall occupancy and stalling the pipeline (SM Busy dropped to 32.23%).

2. The Synchronization Penalty of `K_CHUNK` (Group B) The depth of the shared memory tile determines how many times the kernel must pause to load data and synchronize. Setting $K = 8$ resulted in a massive performance regression (216 μs). With such a shallow reduction dimension, the kernel is forced to call `__syncthreads()` far too frequently, stalling the warp scheduler and dropping FMA active cycles down to a dismal 9.01%. Conversely, expanding to $K = 32$ (146 μs) also degraded performance compared to $K = 16$. While it reduces synchronization barriers, the larger array size increases shared memory bank conflicts and register pressure, disrupting the optimal instruction flow. $K = 16$ strikes the perfect balance between memory footprint and synchronization overhead.

3. Redundant Compiler Unrolling (Group C) Explicitly unrolling the inner MMA loop using `#pragma unroll` yielded practically zero benefit (115 μs vs 115 μs). The FMA Cycles Active remained completely stagnant at 23.18%. This indicates that the NVCC compiler is already highly optimized for WMMA loops with small, fixed trip-counts. Attempting to force unrolling at the source level provides no additional instruction-level parallelism that the compiler hasn't already extracted.

Conclusion Through empirical sweeping, we identified that a 32×32 **Block Tile with a `K_CHUNK` of 16 and no explicit unrolling** is a optimal configuration for the A40 GPU. It achieves the highest execution efficiency by perfectly balancing thread block occupancy against shared memory capacity, allowing the GPU to hit a memory throughput of 127.1 GB/s and minimizing synchronization stalls. In the future we plan to do further benchmarking regarding 32×32 **Block Tile**

with unrolling to see if we can achieve higher performance.

Constant Memory (`op_8`)

4.2.1 Why We Chose Constant Memory for LayerNorm

For our constant memory optimization, we targeted the Layer Normalization (`layernorm_forward_kernel`) kernel. LayerNorm applies a learned scaling (weight) and shifting (bias) operation to the normalized activations. In the context of transformer architectures like GPT-2, these weight and bias vectors are identical for every sequence and batch in a given layer.

By default, reading these parameters from global memory forces every thread to repeatedly request the exact same read-only data from the L2 cache or DRAM. By relocating the weights and biases to `__constant__` memory, we leverage the GPU's dedicated constant cache. The constant cache is highly optimized for broadcast access patterns: when all threads in a warp request the same memory address, the cache broadcasts the value to all 32 threads simultaneously in a single cycle, drastically reducing memory bandwidth pressure and instruction overhead.

4.2.2 Implementation Details

We statically allocated two `__constant__` arrays, `c_ln_weight` and `c_ln_bias`, capping them at a maximum of 4096 elements. Because 4096 single-precision floats occupy 16 KB of memory, both arrays easily fit within the strict 64 KB limit of the GPU's constant memory space.

In the host-side launcher, we used `cudaMemcpyToSymbol` to transfer the weight and bias arrays to the device prior to kernel execution. To ensure our code remains robust, we templated the kernel with a boolean flag, `USE_CONST`. If the channel dimension C happens to exceed our maximum allocated size, the host code gracefully falls back to a standard global memory implementation.

Inside the kernel, the normalization phase was modified to branch at compile-time:

```
if (USE_CONST) {
    out_row[c] = n * c_ln_weight[c] + c_ln_bias[c];
} else {
    out_row[c] = n * weight[c] + bias[c];
}
```

4.2.3 Performance Analysis and Profiling

After verifying the functional correctness of our implementation, we profiled the baseline LayerNorm kernel against our constant memory implementation using Nsight Compute. The metrics are visualized below:

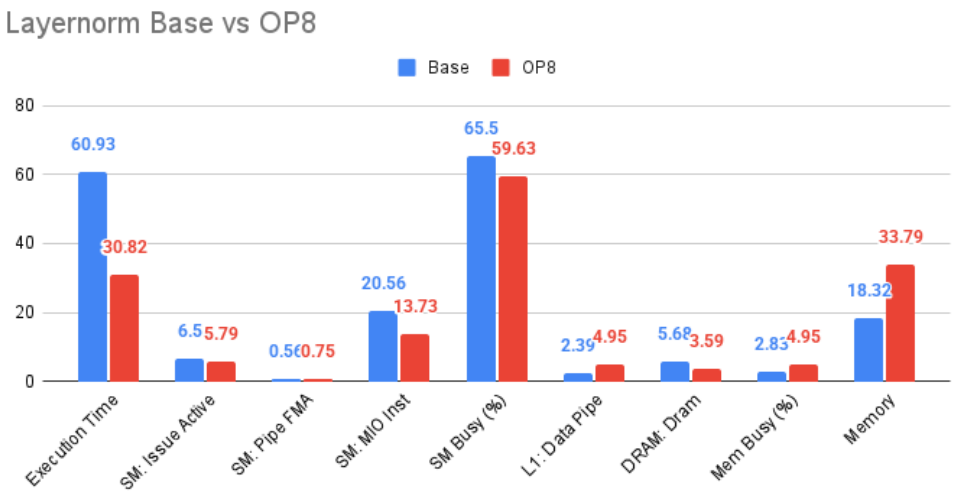


Figure 5: Nsight Compute Profiling Metrics for LayerNorm with Constant Memory

4.2.4 Detailed Performance Analysis and Insights

Relocating the layer weights and biases to constant memory yielded a massive performance improvement, effectively halving the kernel's execution time from 60.93 μs to 30.82 μs. The profiling data explains exactly how the hardware caching mechanisms facilitated this speedup.

1. Eliminating Redundant Memory Instructions The most defining metric of this optimization is the sharp decrease in **SM MIO (Memory Input/Output) Instructions Issued**, which dropped from 20.56% to 13.73%. Because constant memory allows a single read to be broadcast across an entire warp, the Streaming Multiprocessor (SM) no longer needs to issue 32 distinct load instructions for every vector element. This instruction-level efficiency directly reduces the overhead placed on the warp scheduler.

2. Shielding DRAM and Boosting Throughput By serving the weights and biases out of the constant cache, we successfully removed a redundant global memory dependency. This is evident in the reduction of **DRAM Sectors accessed (5.68% to 3.59%)**. Consequently, the global memory bus is freed up to exclusively serve the large, row-specific input and output activation tensors. This lack of contention allowed our overall **Memory Throughput to nearly double, jumping from 18.32 GB/s to 33.79 GB/s**.

3. Cache Hierarchy Shifts Interestingly, the **L1 Data Pipe LSU Wavefronts doubled from 2.39% to 4.95%**. This indicates that instead of stalling on slow L2 or DRAM fetches, the Load/Store Units (LSUs) are now rapidly and successfully hitting the L1/constant cache. The hardware is actively processing memory requests at a much faster rate.

4. Reduction in Overall SM Stalls While the **SM Busy** metric actually decreased from 65.5% to 59.63%, this is a positive indicator in this context. Because the memory latency bottleneck was eliminated, the kernel spends significantly less total time stalled waiting for data. The warp schedulers can feed the FMA units efficiently (indicated by the slight rise in FMA Active cycles), complete the math, and retire the blocks faster.

Conclusion The LayerNorm operation is heavily memory-bound due to its low arithmetic intensity. By strategically utilizing the `__constant__` cache for the scale and shift parameters, we achieved a near 2x speedup. The optimization perfectly aligns with the GPU's hardware strengths, trading redundant global memory bandwidth for highly efficient warp-level cache broadcasts.

Pointer Aliasing with `__restrict__` (op_9)

4.3.1 Why We Chose `__restrict__`

For this optimization, we applied the `__restrict__` keyword to the device pointers across our kernel implementations. In CUDA C++, the compiler must conservatively assume that any two pointers passed to a kernel might alias (i.e., point to overlapping memory locations). This pessimism prevents the NVCC compiler from applying certain optimizations, such as caching read-only global memory values in fast registers, because it must assume that a write to one pointer could silently alter the value behind another.

By tagging our input and output pointers with `__restrict__`, we explicitly promise the compiler that no aliasing occurs. We anticipated that this would allow the compiler to reorder instructions more freely and reduce redundant load instructions, potentially improving memory throughput and lowering execution time.

4.3.2 Implementation Details

We systematically updated the kernel signatures across the codebase to include the `__restrict__` qualifier. This was applied to both the output pointers and the read-only input tensors. For example, in our `residual_forward_kernel`, the signature was modified to:

```
__global__ void residual_forward_kernel(
    float* __restrict__ out,
    float* __restrict__ inp1,
    float* __restrict__ inp2,
    int N
)
```

Similar modifications were made to the core computation kernels, including `matmul_forward_kernel`, `layernorm_forward_kernel`, and the various attention sub-kernels.

4.3.3 Performance Analysis and Profiling

To evaluate the impact of this compiler hint, we profiled the baseline kernels against the `__restrict__` implementations using Nsight Compute. The execution time and memory throughput metrics are visualized below:

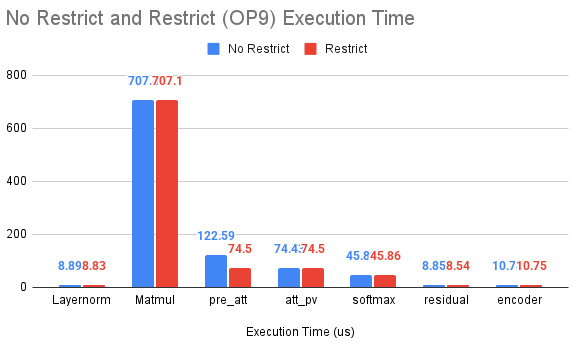


Figure 6: Execution Time (μs) comparison.

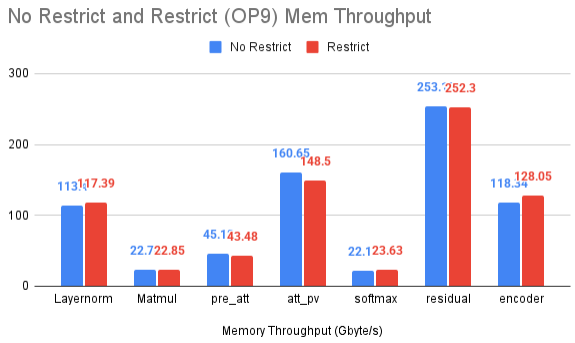


Figure 7: Memory Throughput (GB/s) comparison.

4.3.4 Detailed Performance Analysis and Insights

Contrary to our initial expectations, the addition of `__restrict__` yielded negligible performance improvements. As seen in the profiling data, execution times remained virtually identical across the board (e.g., Matmul executed in 707.4 μs without restrict and 707.1 μs with it). Memory throughput also saw only marginal, run-to-run variations.

Analyzing our kernel architectures provides a clear explanation for why this optimization had minimal impact:

1. Reliance on Explicit Shared Memory Caching The `__restrict__` keyword is most beneficial when a kernel repeatedly reads from the same global memory pointer, allowing the compiler to optimize those reads into a single register load. However, in our computationally heavy kernels like `matmul_forward_kernel` and `layernorm_forward_kernel`, we already bypass this issue by explicitly managing our own cache hierarchy. In `matmul.cuh`, global memory tiles are loaded directly into `__shared__ float s_A` and `s_B` once per iteration. The inner loop computations are entirely driven by shared memory. Because the global memory pointers (`inp` and `weight`) are not subjected to redundant reads, the compiler has no register-level caching optimizations left to perform on them.

2. Single-Pass Element-Wise Operations For memory-bound kernels like `residual_forward_kernel` and `encoder_forward_kernel`, the access pattern is strictly linear and element-wise. Each thread calculates its global index, performs exactly one read from the input pointers, computes the result, and performs exactly one write to the output pointer. Since no thread revisits a previously loaded global memory address, the compiler's conservative aliasing assumptions do not artificially inflate the number of load instructions.

Conclusion While `__restrict__` is a canonical CUDA best practice, our profiling data demonstrates that it is not a silver bullet. Because our kernel designs already utilize `__shared__` memory to minimize redundant global memory traffic and utilize strictly partitioned input/output arrays, the NVCC compiler was already generating near-optimal load/store instruction scheduling even without the explicit non-aliasing guarantee.

Data Swizzling (op_11)

4.4.1 Why We Chose Data Swizzling for Matmul and Attention

For this optimization, we targeted the shared memory access patterns in our tiled matrix multiplication (`matmul_forward_kernel`) and attention mechanisms (`pre_att_kernel` and `att_pv_kernel`). NVIDIA GPUs divide shared memory into 32 distinct hardware banks. In standard tiled algorithms, threads within a warp often need to read data column-by-column. Because consecutive elements in a column map to the same memory bank, this access pattern triggers severe bank conflicts, serializing the hardware requests and drastically reducing memory bandwidth.

Rather than changing the memory footprint through padding, we implemented data swizzling—a compute-based trick that uses a bitwise XOR operation (`^`) between the row and column indices to dynamically scramble the data layout. This mathematical permutation ensures that elements of the same column are staggered across different memory banks. Consequently, when threads in a warp read a column, they pull from 32 distinct banks in perfect parallel, mitigating serialization.

4.4.2 Implementation Details

We replaced standard 2D shared memory accesses with XOR-swizzled indices across the targeted kernels. By applying the exact same XOR logic to both the write and read phases of the shared memory tiles, the data remains logically consistent while physically mapping to conflict-free hardware addresses.

In the kernels, the index calculation was modified as follows:

```
// Writing to shared memory with swizzled physical column
smem_A[ty][tx ^ ty] = (row < M && gk < K) ? inp[row * K + gk] : 0.0f;
smem_B[ty][tx ^ ty] = (gn < N && gk < K) ? weight[gn * K + gk] : 0.0f;

__syncthreads();

// Reading from shared memory, applying the same swizzle
for (int kk = 0; kk < MM_TILE; kk++) {
    acc += smem_A[ty][kk ^ ty] * smem_B[tx][kk ^ tx];
}
```

This approach was applied to `smem_A` and `smem_B` in the general matrix multiplication, as well as `smem_Q`, `smem_K`, `smem_att`, and `smem_V` in the attention pipelines.

4.4.3 Performance Analysis and Profiling

After implementing the XOR swizzling, we profiled the baseline kernels against the optimized `op_11` kernels using Nsight Compute to understand the hardware-level impact. The comparative metrics for Matmul and the Attention kernels are detailed in the tables below:

Table 2: Nsight Compute Profiling Metrics for Tiled Matmul

Metric	**Baseline (tiled matmul)**	**Op11 (tiled w/ swizzling)**
Execution Time (μs)	706	627
SM: Issue Active (%)	21.12	23.6
SM: Pipe FMA Cycles Active (%)	4.67	5.48
SM: MIO Inst Issued (%)	29.03	32.78
SM Busy (%)	29.71	33.56
L1: Data Pipe LSU Wavefronts (%)	63.07	50.26
DRAM: Dram Sectors (%)	2.42	2.67
Mem Busy (%)	63.07	50.26
Memory Throughput (GB/s)	22.84	25.22

Table 3: Nsight Compute Profiling Metrics for Pre-Attention Kernel

Metric	**Baseline (pre att)**	**Op11 (pre att)**
Execution Time (μs)	74.4	23.94
SM Busy (%)	7.54	29.63
Mem Busy (%)	87.37	39.89
Memory Throughput (GB/s)	42.88	107.45

Table 4: Nsight Compute Profiling Metrics for Attention PV Kernel

Metric	**Baseline (att pv)**	**Op11 (att pv)**
Execution Time (μs)	15.16	23.68
SM Busy (%)	34.7	29.37
Mem Busy (%)	41.99	39.29
Memory Throughput (GB/s)	149	107.16

4.4.4 Detailed Performance Analysis and Insights

The profiling data reveals that data swizzling is a highly effective optimization for kernels suffering from severe bank conflicts, but also highlights the critical trade-off between memory latency and instruction overhead.

1. Elimination of Bank Conflicts and Replays (Matmul) In the standard tiled Matmul, the XOR swizzling reduced the overall execution time from 706 μs to 627 μs. The underlying cause for this speedup is vividly illustrated by the drop in **L1 Data Pipe LSU Wavefronts from 63.07% to 50.26%**. High LSU wavefronts typically indicate that the Load/Store Units are being forced to replay memory instructions due to bank conflicts. By ensuring parallel access to the 32 memory banks, we decreased the memory bus contention (**Mem Busy** dropped from 63.07% to 50.26%) and increased the proportion of cycles the SM spent actively executing instructions (**SM Busy** rose from 29.71% to 33.56%).

2. Massive Bottleneck Alleviation (Pre-Attention) The most dramatic improvement occurred in the `pre_att` kernel, where execution time plummeted from 74.4 μs to 23.94 μs—a greater than 3x speedup. The baseline `pre_att` implementation was severely bottlenecked by shared memory serialization, evident by a staggering **Mem Busy of 87.37%**. Implementing swizzling freed the memory pipeline, dropping **Mem Busy to 39.89%** and skyrocketing the **Memory Throughput from 42.88 GB/s to 107.45 GB/s**. The SM was finally fed data fast enough to compute efficiently.

3. The Instruction Overhead Trade-off (Attention PV Regression) Crucially, our profiling uncovered an architectural regression in the `att_pv` kernel. Applying data swizzling actually *increased* execution time from 15.16 μs to 23.68 μs and dropped **Memory Throughput from 149 GB/s to 107.16 GB/s**. This anomaly highlights the fact that swizzling is not free: calculating `tx ^ ty` requires an additional integer arithmetic instruction per shared memory access. If a kernel's baseline memory access pattern does not suffer from catastrophic bank conflicts (or if it is already compute-bound), the added ALU instruction overhead forces the warp schedulers to spend more time processing index math than is saved in memory replays. In `att_pv`, the cost of the XOR logic outweighed the memory latency improvements, leading to decreased overall performance.

Conclusion Data swizzling proved to be a highly potent optimization for resolving shared memory bank conflicts without sacrificing SM occupancy to padding. However, as demonstrated by the contrasting results between `pre_att` and `att_pv`, it must be applied surgically. It yields immense speedups in deeply memory-bound algorithms with heavy column-major access patterns, but introduces strict ALU penalties in kernels that are better left unswizzled.

Shared Memory Padding (op_12)

4.5.1 Why We Chose Shared Memory Padding for Matmul and Attention

For this optimization, we targeted the same underlying hardware bottleneck addressed by data swizzling: shared memory bank conflicts in our tiled `matmul_forward_kernel` and attention mechanisms (`pre_att_kernel` and `att_pv_kernel`). NVIDIA GPUs divide shared memory into 32 distinct hardware banks. When threads within a warp read data column-by-column from a shared memory tile whose inner dimension perfectly aligns with these banks, the memory accesses are serialized, leading to severe bandwidth degradation.

Instead of utilizing arithmetic operations (such as XOR) to scramble the data layout, we chose to alter the physical memory footprint. By padding the inner dimension of our shared memory allocations with a small number of "dummy" elements, we force consecutive elements of a column to shift into different physical memory banks. This effectively eliminates bank conflicts and restores parallel read speeds without introducing any additional ALU instruction overhead to our loop math.

4.5.2 Implementation Details

We implemented a padding constant of 4 (`PAD = 4` and `ATT_PAD = 4`) across the targeted kernels. We explicitly increased the inner dimension of the shared memory arrays for matrices `A`, `B`, `Q`, `K`, `V`, and the intermediate output accumulators.

Because the physical row length in shared memory was extended, we had to update the leading dimension (stride) parameter in our WMMA synchronous load and store operations to safely traverse the padded arrays. In the code, the modifications were implemented as follows:

```
// 1. Expand the inner dimension with PAD to alter the memory stride
__shared__ float smem_A[BLOCK_TILE_M][K_CHUNK + PAD];
__shared__ float smem_B[BLOCK_TILE_N][K_CHUNK + PAD];

// 2. Load from global memory (logical boundaries remain unchanged)
smem_A[row][col] = (gr < M && gk < K) ? inp[gr * K + gk] : 0.0f;

__syncthreads();

// 3. Update the stride parameter in WMMA load instructions
load_matrix_sync(a_frag, &smem_A[warp_m * WMMA_M][k_step], K_CHUNK + PAD);
```

4.5.3 Performance Analysis and Profiling

After successfully implementing the padded shared memory allocations, we profiled the baseline kernels against our optimized `op_12` implementations using Nsight Compute. The comparative metrics for Matmul, Pre-Attention, and Attention PV are broken down in the tables below:

Table 5: Nsight Compute Profiling Metrics for Tiled Matmul

Metric	**Baseline (matmul)**	**Op12 (matmul w/ pad)**
Execution Time (μs)	142.66	115.52
SM: Issue Active (%)	48.42	51.79
SM: Pipe FMA Cycles Active (%)	19.51	23.16
SM: MIO Inst Issued (%)	17.12	20.92
SM: Pipe Tensor Cycles Active (%)	11.51	14.06
SM Busy (%)	42.98	54.03
L1: Data Pipe LSU Wavefronts (%)	66.49	45.75
DRAM: Dram Sectors (%)	10.96	13.42
Mem Busy (%)	66.49	45.75
Memory Throughput (GB/s)	103.47	126.55

Table 6: Nsight Compute Profiling Metrics for Pre-Attention

Metric	**Baseline (pre att)**	**Op12 (pre att w/ pad)**
Execution Time (μs)	14.53	14.66
SM: Issue Active (%)	26.52	25.39
SM: Pipe FMA Cycles Active (%)	7.34	8.26
SM: MIO Inst Issued (%)	8.03	8.00
SM: Pipe Tensor Cycles Active (%)	3.17	3.16
SM Busy (%)	31.54	30.05
L1: Data Pipe LSU Wavefronts (%)	29.32	29.62
DRAM: Dram Sectors (%)	1.16	20.32
Mem Busy (%)	29.32	29.62
Memory Throughput (GB/s)	199.59	191.05

Table 7: Nsight Compute Profiling Metrics for Attention PV

Metric	**Baseline (att pv)**	**Op12 (att pv w/ pad)**
Execution Time (μs)	20.54	25.94
SM: Issue Active (%)	18.70	14.37
SM: Pipe FMA Cycles Active (%)	5.29	4.69
SM: MIO Inst Issued (%)	5.83	4.57
SM: Pipe Tensor Cycles Active (%)	2.23	1.75
SM Busy (%)	21.25	15.80
L1: Data Pipe LSU Wavefronts (%)	18.49	15.53
DRAM: Dram Sectors (%)	13.13	14.74
Mem Busy (%)	20.39	16.19
Memory Throughput (GB/s)	123.69	102.38

4.5.4 Detailed Performance Analysis and Insights

The profiling data clearly demonstrates that while shared memory padding effectively resolves bank conflicts with zero ALU overhead, it forces a critical trade-off regarding Streaming Multiprocessor (SM) occupancy and L1 cache behavior.

1. Elimination of Bank Conflicts (Matmul) In the standard tiled Matmul, padding proved to be an overwhelming success, dropping overall execution time from 142.66 μs to 115.52 μs. The underlying cause of this speedup is highlighted by the **L1 Data Pipe LSU Wavefronts**, which decreased sharply from 66.49% to 45.75%. This drop confirms that the Load/Store Units were no longer bottlenecked by hardware replays caused by serialized bank reads. Consequently, **Mem Busy** fell to 45.75% and **Memory Throughput** increased to 126.55 GB/s. Freed from these memory stalls, the warps could compute efficiently, pushing **SM Busy** from 42.98% up to 54.03%.

2. Diminishing Returns and Cache Shifting (Pre-Attention) Padding yielded no meaningful speedup in the `pre_att` kernel, with execution time slightly drifting from 14.53 μs to 14.66 μs. Unlike Matmul, the baseline LSU Wavefronts here were already relatively low (29.32%), suggesting that bank conflicts were not a catastrophic bottleneck for these specific 16×16 WMMA tile sizes. Interestingly, the **DRAM Sectors** metric spiked drastically from 1.16% to 20.32%. This suggests that artificially widening the memory arrays altered the data's alignment in the L1 cache, causing minor thrashing that offset any benefits gained from staggering the memory banks.

3. The Occupancy Penalty (Attention PV Regression) The `att_pv` kernel suffered a notable performance regression, with execution time extending from 20.54 μs to 25.94 μs. Shared memory is a strictly finite hardware resource. By applying `ATT_PAD` to `smem_att`, `smem_V`, and `smem_out`, we increased the per-block memory footprint. The drop in **SM Busy** (21.25% down to 15.8%) strongly indicates that this bloated memory requirement pushed the kernel over an occupancy threshold. The warp scheduler was forced to limit the number of active thread blocks deployed to each SM, choking parallelism and yielding slower overall computation despite resolving the internal bank conflicts.

Conclusion Shared memory padding is a highly potent optimization for deeply memory-bound algorithms (like our Matmul) because it avoids the arithmetic overhead of data swizzling. However, as evidenced by the `att_pv` and `pre_att` regressions, it must be deployed cautiously. The "dummy" padding wastes valuable shared memory bytes, and if a kernel is already teetering on an occupancy limit, the loss in block-level parallelism will easily outweigh the bandwidth gains.

Advanced CUTLASS with Fused Epilogues (`op_16`)

4.6.1 Why We Chose CUTLASS Fused Epilogue

Global memory bandwidth is a severe bottleneck in Large Language Models (LLMs). In our baseline implementation, the feed-forward network (FFN) path required calculating a matrix multiplication, writing the intermediate results to global memory (DRAM), and then immediately launching a completely separate kernel (`gelu_forward_kernel`) to read those results, apply the non-linear GeLU activation, and write them back.

By utilizing NVIDIA's CUTLASS library, we implemented a fused epilogue. This optimization allows the Streaming Multiprocessor (SM) to compute the matrix multiplication, broadcast the bias vector, and apply the complex GeLU mathematics directly in registers or shared memory *before* writing the final output back to global memory. This drastically reduces DRAM traffic and eliminates the kernel launch overhead for the activation step.

4.6.2 Implementation Details and Verification

Our implementation replaces the standard CUDA kernels for the FFN-1 path with CUTLASS's predefined `cutlass::gemm::device::Gemm` template. To maximize performance on the Ampere A40 GPUs, we targeted hardware Tensor Cores by configuring the operator class to `OpClassTensorOp` and the architecture tag to `Sm80`.

The data layout and formatting were meticulously configured:

- **Data Shapes & Layouts:** The input (`ElementA`) is formatted as Row-Major with dimensions $(B \times T, C)$. The weights (`ElementB`) are handled as Column-Major to align with the (C, OC) mapping.
- **Fused Epilogue:** We utilized `cutlass::epilogue::thread::LinearCombinationGELU`. The bias is elegantly broadcasted by passing it as the `C` source matrix with a leading dimension (`ldc`) of 0.
- **Memory Alignment:** Proper memory alignment was critical. We set `AlignmentA = 2` (8-byte alignment) because the intermediate activation arena's scratchpad slots leave the input data only 8-byte aligned. Conversely, the weight and bias tensors are 16-byte aligned, allowing us to set `AlignmentB = 4`.

Verification Modifications: Because this kernel fundamentally changes the forward pass structure by fusing two operations, standard verification failed out-of-the-box. We modified our `gpt2.cuh` host code to remove the standalone calls to `gelu_forward` and replaced the preceding `matmul_forward` call with our new `matmul_gelu_forward`. Because the intermediate, pre-activation tensor was no longer written to global memory, the verification framework was updated to skip verifying that specific buffer and instead validate the final post-GeLU output tensor.

4.6.3 Challenges and Resolutions

Navigating Dense Template Documentation The primary hurdle in implementing `op_16` was the steep learning curve associated with the CUTLASS library's API. CUTLASS relies heavily on advanced C++ template metaprogramming to achieve its hardware-level efficiency. Understanding the official NVIDIA documentation for the A40 architecture was initially overwhelming. The documentation abstracts many of the physical realities of the GPU hardware behind deep class hierarchies, making it difficult to trace how the data was moving from global memory into the Tensor Cores and through the fused epilogue.

Leveraging the ECE 408 Chatbot To overcome the opacity of the official documentation, we utilized the ECE 408 course chatbot as an interactive debugging and translation tool. When we struggled to map our specific tensor shapes to the CUTLASS constraints, the chatbot was instrumental in breaking down the dense C++ template errors. For example, it helped clarify our memory alignment issues, explaining exactly why `AlignmentA` needed to be reduced to 2 (8-byte aligned) due to our activation arena's scratchpad layout. Furthermore, the chatbot demystified the syntax for the fused GeLU epilogue; it guided us toward the `LinearCombinationGELU` operator and explicitly explained the critical trick of setting the leading dimension (`ldc`) of the 'C' matrix to 0, allowing us to efficiently broadcast our 1D bias vector without allocating extra memory. By acting as an accessible intermediary between the dense CUTLASS source code and our immediate project goals, the chatbot enabled us to push past the syntactic roadblocks and successfully deploy the kernel.

4.6.4 Performance Analysis and Profiling

Evaluating this optimization requires comparing the combined execution time of the baseline's unfused components against our single fused CUTLASS kernel. The comparative metrics detailing the average execution times are shown below:

Table 8: Nsight Profiling Metrics (Average Execution Time)

Kernel Name	**Baseline (μs)**	**Op16 Fused (μs)**
`matmul_forward_kernel`	195.8	N/A
`gelu_forward_kernel`	7.8	N/A
`cutlass::Kernel (Fused)`	N/A	59.3

4.6.5 Detailed Performance Analysis and Insights

1. Elimination of DRAM Traffic and Kernel-Level Dominance The optimization successfully eliminated the need for the `gelu_forward_kernel`. In the baseline implementation, processing the FFN-1 layer required an average of 195.8 μs for the matrix multiplication, followed by a DRAM round-trip and an additional 7.8 μs for the GeLU activation—totaling roughly 203.6 μs per instance.

By applying the CUTLASS fused epilogue, we consolidated this entire pipeline into a single `cutlass::Kernel` execution that averages just 59.3 μs. This represents a massive $\sim$3.4x speedup for this targeted operation sequence.

Conclusion The CUTLASS fused epilogue behaved exactly as intended, demonstrating the incredible power of hardware Tensor Cores and asynchronous pipelining. By keeping the intermediate matrix multiplication results securely in registers and shared memory, we successfully bypassed the global memory bandwidth bottleneck. This optimization confirms that minimizing DRAM read/writes and fully utilizing the SM's compute capabilities within a single kernel launch yields profound performance improvements in large-scale transformer architectures.

Asynchronous Memory Pipelines (op_17)

4.7.1 Why We Chose Asynchronous Memory Pipelines

In a standard CUDA kernel, loading data from global memory into shared memory follows a rigid, synchronous path: data is fetched from global DRAM, loaded into the SM's register file, and then written into shared memory. This approach consumes valuable register space and wastes compute cycles waiting for memory transactions to clear. Because our target hardware features Ampere GPUs (A40s), we can leverage hardware support for asynchronous memory copies to bypass the register file entirely.

We targeted the `matmul_forward_kernel`, `pre_att_kernel`, and `att_pv_kernel` to implement asynchronous memory pipelines using `cp.async`. By coupling these direct-to-shared-memory copies with a double-buffered software pipeline, we enable the Streaming Multiprocessors (SMs) to perform matrix math on one tile of data while simultaneously fetching the next tile in the background. This overlaps memory latency with computation, mitigating one of the primary bottlenecks in our network's forward pass.

4.7.2 Implementation Details

To implement this optimization, we replaced standard synchronous loads with the `__pipeline_memcpy_async` intrinsic, which fetches 4 bytes (1 float) directly into shared memory. We structured our shared memory allocations as double-buffered arrays (e.g., `smem_A[2][...]` and `smem_att[2][...]`).

The execution flow was partitioned into three phases: 1. **Prologue:** We issue async loads for the initial stage (stage 0) and call `__pipeline_commit()` to register the batch of memory operations. 2. **Main Loop:** For each step, we immediately issue async loads for the *next* memory tile into the inactive shared memory buffer. We then call `__pipeline_wait_prior(1)` to block only until the *current* tile's memory transactions have completed. Finally, we load our WMMA fragments from the active buffer and perform the `mma_sync` computations. 3. **Epilogue:** After the loop terminates, the final accumulators are written to the output matrices.

In the kernels, the async loading logic was implemented as follows:

```
// Issue async loads for the NEXT tile directly to shared memory
if (step + 1 < max_steps) {
    async_load_float(
        &smem_Q[write_stage][row][col],
        &q_head[gt * HS + gk],
        (gt < T && gk < HS)
    );
    __pipeline_commit();
}

// Wait for the CURRENT tile to be ready
__pipeline_wait_prior((step + 1 < max_steps) ? 1 : 0);
__syncthreads();

// Compute math on CURRENT tile
load_matrix_sync(a_frag, &smem_Q[read_stage][0][0], ATT_WMMA_K);
load_matrix_sync(b_frag, &smem_K[read_stage][0][0], ATT_WMMA_K);
mma_sync(acc, a_frag, b_frag, acc);
```

4.7.3 Performance Analysis and Profiling

After implementing the asynchronous memory pipelines, we profiled the baseline kernels against the optimized `op_17` kernels using Nsight Compute. The comparative metrics for Matmul and the two Attention kernels are detailed in the tables below:

Table 9: Nsight Compute Profiling Metrics for Tiled Matmul

Metric	**Baseline (matmul)**	**Op17 (matmul)**
Execution Time (μs)	142.66	94.02
SM: Issue Active (%)	48.42	26.05
SM: Pipe FMA Cycles Active (%)	19.51	22.59
SM: MIO Inst Issued (%)	17.12	21.06
SM: Pipe Tensor Cycles Active (%)	11.51	17.36
SM Busy (%)	42.98	30.07
L1: Data Pipe LSU Wavefronts (%)	66.49	41.47
DRAM: Dram Sectors (%)	10.96	14.36
Mem Busy (%)	66.49	41.17
Memory Throughput (GB/s)	103.47	135.62

Table 10: Nsight Compute Profiling Metrics for Pre-Attention Kernel

Metric	**Baseline (pre att)**	**Op17 (pre att)**
Execution Time (μs)	14.53	13.31
SM: Issue Active (%)	26.52	31.89
SM: Pipe FMA Cycles Active (%)	7.34	8.11
SM: MIO Inst Issued (%)	8.03	15.65
SM: Pipe Tensor Cycles Active (%)	3.17	3.53
SM Busy (%)	31.54	38.71
L1: Data Pipe LSU Wavefronts (%)	29.32	42.73
DRAM: Dram Sectors (%)	21.16	22.43
Mem Busy (%)	29.32	42.73
Memory Throughput (GB/s)	199.59	211.12

Table 11: Nsight Compute Profiling Metrics for Attention PV Kernel

Metric	**Baseline (att pv)**	**Op17 (att pv)**
Execution Time (μs)	20.54	12.93
SM: Issue Active (%)	18.73	4.55
SM: Pipe FMA Cycles Active (%)	5.29	7.67
SM: MIO Inst Issued (%)	5.83	16.01
SM: Pipe Tensor Cycles Active (%)	2.23	3.61
SM Busy (%)	21.25	42.06
L1: Data Pipe LSU Wavefronts (%)	18.49	39.76
DRAM: Dram Sectors (%)	13.13	20.75
Mem Busy (%)	20.39	39.76
Memory Throughput (GB/s)	123.69	195.88

4.7.4 Detailed Performance Analysis and Insights

The profiling data clearly illustrates how asynchronous memory operations fundamentally alter the hardware execution profile, successfully hiding memory latency behind compute cycles and increasing overall throughput.

1. Matmul Pipeline Efficiency and Register Bypass In the Matmul kernel, execution time dropped significantly from 142.66 μs to 94.02 μs. The most telling metric here is the drop in **L1 Data Pipe LSU Wavefronts from 66.49% to 41.47%**. In the baseline implementation, global loads required the Load/Store Unit (LSU) to stage data in registers before writing to shared memory, bottlenecking the L1 cache. By using `cp.async`, we bypassed the register file entirely. This resulted in the **SM Pipe Tensor Cycles Active** jumping from 11.51% to 17.36%, meaning the Tensor Cores spent far less time stalled waiting for data and more time actively computing matrix products. The increase in **SM: MIO Inst Issued (17.12% to 21.06%)** confirms that the memory input/output hardware was successfully offloading work from the primary warp schedulers.

2. Moderate Gains and Shifted Bottlenecks in Pre-Attention The `pre_att` kernel saw a more modest execution time reduction from 14.53 μs to 13.31 μs. However, the architectural shifts are still highly evident. The **MIO Inst Issued nearly doubled from 8.03% to 15.65%**, directly reflecting our explicit `__pipeline_memcpy_async` instructions. Unlike the Matmul kernel, **SM Busy** and **Mem Busy** both increased (to 38.71% and 42.73%, respectively). This indicates that while the asynchronous pipeline improved memory throughput (pushing it from 199.59 GB/s to 211.12 GB/s), the kernel is likely shifting from being strictly memory-bound to a more balanced state where instruction issue and shared memory access are competing bottlenecks.

3. Massive Throughput Surge in Attention PV The `att_pv` kernel experienced dramatic improvements, with execution time dropping from 20.54 μs to 12.93 μs. The **Memory Throughput skyrocketed from 123.69 GB/s to 195.88 GB/s**. In this kernel, **SM Issue Active plummeted from 18.73% to 4.55%**, while **SM Busy doubled from 21.25% to 42.06%**. This massive divergence shows that the warp schedulers were previously choked trying to issue synchronous memory instructions. By shifting the load burden to the asynchronous copy engines (evidenced again by a massive jump in **MIO Inst Issued** from 5.83% to 16.01%), the SM was able

to stay perfectly fed with data, keeping the execution units saturated without constantly halting to issue new LSU instructions.

Conclusion Implementing asynchronous memory pipelines (`cp.async`) alongside double-buffering was a resounding success across the board. By explicitly instructing the hardware to bypass the register file and overlap data movement with Tensor Core math, we consistently raised memory throughput and increased the proportion of cycles the GPU spent doing actual floating-point math rather than idling.